

ТЗ: AI-бот поддержки QuickOffer

- > Короткая версия для внешнего разработчика и технического дополнения Максимом.
- > Бот подключается к текущему @quick_offer_help, отвечает клиентам, исполняет шесть типовых флоу и передаёт сложные кейсы человеку.

1. Что нужно построить

Бот работает в двух режимах.

Режим А — детерминированные флоу

1. Возврат платежа и удаление подписки.
2. Карьерная помощь.
3. Архивация вакансии в /jobs.
4. Реферальный промокод.
5. Промокод за отзыв.
6. Оплата криптовалютой/иностранной картой.

Для этих запросов используются серверные state machines: бот собирает данные, проверяет eligibility, показывает preview, запрашивает подтверждение и вызывает строго определённый backend tool.

Режим В — остальные вопросы

LLM расследует запрос через approved knowledge base и read-only данные аккаунта. Затем:

- отвечает пользователю;
- задаёт уточнение;
- либо передаёт человеку summary с уже собранными фактами.

Главное ограничение

LLM — интерпретатор, не субъект авторизации и не исполнитель мутаций.

Запрещено давать модели:

- production DB credentials;
- raw SQL/shell;
- generic write tool;
- произвольный HTTP;
- возможность выбирать user ID, сумму, тариф, скидку или роль как источник истины.

Все изменения выполняются через узкие internal APIs с policy, idempotency, audit и readback.

2. Что есть сейчас

Аудит на 18.08.2026:

Система	Стек и роль	Что важно для бота
fuckhr/fuckhr-api	Go, Gin, GORM, PostgreSQL	основной пользовательский backend; внутри уже есть cmd/ops и internal/usecase/ops
fuckhr/fuckhr-ui	Next.js, React, TypeScript	FAQ, оферта, формы отзывов, payment UI
fuckhr/jobs-api	Django, DRF, PostgreSQL	отдельный каталог /jobs; публичный API read-only, управление через Django Admin
fuckhr/ci-cd	GitHub Actions + Docker Compose	dev/prod deploy workflows

Проверенные ревизии:

- fuckhr-api: 0602e4f626b8ff2d4ca3484ca6a2ad145a8935d6;
- fuckhr-ui: de7f7f2fad82dd0157fe78c50cffc6b94256bcf3;
- jobs-api: 6bea5d40e4c4de858b6c20a205bb2344c7c8e110.

Перед разработкой нужно обновить SHA.

Что уже умеет ops backend

- найти пользователя;
- посмотреть платежи/подписку/поиски;
- выдать, продлить, изменить и soft-delete подписку;
- отключить автопродление;
- создать/изменить/удалить промокод;
- записать admin_action audit.

Чего нет

- M2M support API;
- backend refund flow — возврат делается вручную через YooKassa;
- API для архивации /jobs;
- actor attribution/idempotency/approval chain для бота;
- безопасного runtime-RAG для поддержки.

Текущую HTML/htmx ops-панель нельзя использовать как API и нельзя парсить ботом.

3. Архитектура

```
Telegram @quick_offer_help
|
v
support-bot service
- Telegram adapter
- conversation state
- intent router
- deterministic flow engine
- LLM investigation
- staff handoff/approval
- без production DB credentials
|
+--> private M2M --> fuckhr-api supportapi
| - identity/context read
| - subscription/promo actions
| - payment/refund requests
| - audit/readback
|
+--> private M2M --> jobs-api supportapi
| - job lookup
| - persistent suppression
|
+--> internal Telegram chat
- tickets
- approve/deny/request info
- manual replies
```

Рекомендуется отдельный support-bot deploy unit. Он хранит только диалоги/actions и вызывает internal APIs. Domain data остаются в своих сервисах.

Telegram integration

[TODO MAX] определить тип @quick_offer_help:

- Bot API bot — переиспользовать token, но оставить только одного consumer updates;
- Telegram Business account — подключить business bot и учитывать business_connection_id;
- обычный user account — сначала перейти на Bot API/Business integration.

Userbot под личной сессией — не default.

4. Identity и роли

Клиент

Основной матч:

```
Telegram sender ID → users.tg_id → QuickOffer user
```

Если Telegram не привязан:

1. бот создаёт одноразовую ссылку;
2. клиент открывает её в авторизованной QuickOffer web session;

3. backend связывает conversation с user ID;
4. token одноразовый и short-lived.

Email, username, screenshot или написанный вручную ID не подтверждают владение аккаунтом.

Сотрудник

[TODO MAX]:

- approval chat/topic;
- staff Telegram ID allowlist;
- роли support, finance, admin;
- дежурные и SLA.

Approval привязывается к staff ID, action ID и hash точных параметров. Изменение суммы/пользователя/тарифа аннулирует approval.

5. Риск и автономность

Действие	V1
Public FAQ	автоматически
Персональный read-only ответ	автоматически после identity
Создать карьерный ticket	автоматически
Создать/вернуть referral promo	автоматически
Создать review promo	автоматически после backend eligibility
Отправить alt-payment реквизиты	автоматически из secret service
Выдать подписку после alt-payment	только staff approve
Архивировать вакансию	только staff approve
Refund + удалить подписку	всегда staff approve
Bulk/partial refund/chargeback	только человек, вне автономии v1

6. Флоу 1 — refund и удаление подписки

Policy

Бот не обещает refund. Основания проверяет сервер, решение принимает сотрудник.

Подтверждённые основания:

- услуга ещё не началась;
- двойное/ошибочное списание;
- сервис нельзя использовать по вине QuickOffer.

State machine

```
identify
→ показать последние платежи
→ клиент выбирает платёж
→ backend возвращает payment/subscription/usage preview
→ собрать причину и доказательства
→ показать сумму и последствия удаления подписки
→ подтверждение клиента
→ staff approve/deny/request info
→ refund
→ authoritative provider check
→ revoke subscription/searches
→ receipt клиенту
```

MVP

Текущий backend не умеет refund. Рекомендуемый v1:

1. staff approves;
2. сотрудник делает refund в YooKassa dashboard;
3. вводит provider refund ID;
4. backend перепроверяет payment;
5. после подтверждения soft-delete subscription, деактивирует поиски, останавливает runs и отключает renewal.

Нельзя делать refund изменением `payments.status` в PostgreSQL.

Если provider refund прошёл, а local revoke упал — action переходит в `reconciliation_pending`; provider вызов не повторяется.

7. Флоу 2 — карьерная помощь

Policy

- бот только собирает данные и отправляет команде;
- результат команда присылает вручную;
- текущий формат — письменные рекомендации;
- eligibility: Premium 14 дней, скидка <16%;
- один кейс = одна профессия + одно резюме;
- SLA — до 48 часов после полного комплекта данных.

Что собрать

- профессия/целевая роль;
- резюме/PDF;
- география/формат работы;
- зарплатные ожидания;

- опыт и длительность поиска;
- проблемы и вопросы;
- согласие передать данные эксперту.

State machine

identify → eligibility → intake → summary → confirmation
→ ticket в internal chat → staff take/need info/sent result → close

[TODO MAX]: chat/topic, ответственные, способ ручной отправки результата.

8. Флоу 3 — архивация вакансии /jobs

Собрать:

- URL/slug;
- причину;
- кто обращается — работодатель/правообладатель/кандидат;
- доказательства при необходимости.

lookup job → ticket → staff approve
→ persistent suppress → authoritative readback → receipt

Обычного `archived_at` недостаточно: parser может вернуть вакансию. В `jobs-api` нужен persistent state, например:

- `suppressed_at`;
- `suppression_reason`;
- `ticket_id`;
- `suppressed_by`;
- запрет reactivation для source/company/external ID.

Hard delete не default: он ломает историю, каскады и не защищает от повторного импорта.

9. Флоу 4 — реферальный промокод

Правила:

- один код на QuickOffer account;
- скидка другу 15%;
- бессрочный;
- bot только создаёт/возвращает code;
- cashback остаётся ручным;
- повторный запрос возвращает тот же code.

```
identify → get existing owner mapping
→ found: return code
→ not found: generate unique 15% promo + owner mapping atomically
→ readback → send code/link
```

Нужна отдельная ownership mapping. Нельзя без проверки использовать `promocodes.user_id`: оно может означать «код доступен только этому покупателю», а referral code должны применять друзья.

10. Флоу 5 — промокод за отзыв

Правила Кирилла:

- скидка 15%;
- без срока действия;
- любой sentiment;
- trimmed text >30 символов;
- только backend review verified user;
- один promo/account;
- безопасный default: user-bound, `max_uses=1`.

```
identify → find own eligible review
→ no review: send review form
→ existing promo: return it
→ create 15% no-expiry one-use promo
→ readback → send code
```

Frontend сейчас говорит «Отзыв опубликован», хотя backend создаёт `Published=false`. Бот должен говорить «Отзыв получен».

11. Флоу 6 — crypto/foreign card

Правила:

- реквизиты отправляет бот;
- реквизиты берутся только из secret/config service;
- LLM их не видит и не генерирует;
- staff проверяет платёж;
- staff approves выдачу подписки;
- ручная подписка не имеет auto-renewal.

```
identify → method → live tariff/period → quote
→ verified requisites + anti-fraud warning
→ proof/tx hash → staff verification/approve
→ grant plan/days → manual payment receipt → readback
```

Если у пользователя уже active paid subscription — fail closed и решение сотрудника; нельзя молча заменить её текущим ops grant.

[TODO MAX]:

- методы/кошельки/networks;
- foreign-card provider;
- курс/комиссия/quote TTL;
- fiscal/accounting process;
- verification owner.

12. Остальные вопросы: LLM investigation

Разрешённые read-only tools

- approved FAQ/offer/policies;
- user support snapshot;
- subscription state;
- masked recent payments;
- search health/recent errors/applications;
- current incidents;
- review/referral eligibility;
- job public state.

Curated support KB обязана иметь owner, version, effective date и review date. Нельзя подключать весь Brain без фильтра: там есть drafts и устаревшие планы.

Алгоритм

classify → public/authenticated → minimum retrieval
→ compare with policy → answer/clarify/handoff

Обязательный handoff

- клиент просит человека;
- identity не подтверждена;
- деньги/chargeback/юридическая претензия;
- account takeover/security;
- чужие данные;
- неизвестная policy;
- ambiguous target;
- repeated tool failure;
- provider/local conflict;

- два неуспешных ответа подряд.

После handoff bot не продолжает mutation flow.

13. Internal support API

Новый private transport в fuckhr-api, например internal/transport/supportapi.

Требования:

- mTLS или short-lived signed service token;
- scopes per operation;
- private ingress;
- Idempotency-Key, action ID, ticket ID, trace ID;
- preview/confirm/approve/execute;
- critical params замораживаются в action snapshot;
- execute повторно читает state и отклоняет stale action;
- domain mutation и audit — одна local transaction;
- authoritative readback перед ответом клиенту.

Пример pattern:

```
POST /internal/support/v1/actions/{type}/preview
POST /internal/support/v1/actions/{id}/confirm-user
POST /internal/support/v1/actions/{id}/approve
POST /internal/support/v1/actions/{id}/execute
GET /internal/support/v1/actions/{id}
```

Jobs suppression API должен жить в jobs-api, потому что job — его aggregate.

14. State/audit

Минимальные сущности: conversation, message/attachment, ticket, action, approval и outbox.

Action хранит immutable ID, risk/state, actor/target, frozen params+hash, policy version, idempotency key, approvals, provider ID и reconciliation status. Финальная domain mutation также пишет admin_action/jobs audit с support action ID.

15. Безопасность P0

- LLM investigation физически не имеет mutation tools.
- Нет raw SQL/shell/arbitrary HTTP.
- Identity только из trusted context.
- Object-level authorization на каждом read/write.
- Сумма/тариф/скидка/target вычисляются сервером.
- Approval привязан к exact params и one-time nonce.

- Replay/concurrent request создаёт максимум один effect.
- Пользовательские сообщения/PDF/reviews/vacancy text — untrusted data, не instructions.
- PII минимизируется до LLM и редактируется в logs.
- Separate dev/staging/prod credentials и provider accounts.
- Kill switches по mutations/flow/model.
- Auth/policy/audit outage → fail closed + handoff.

Release blocker:

- 0 unauthorized mutations;
- 0 cross-user disclosures;
- 0 approval bypass;
- 0 duplicate effects в replay/race tests.

16. Тестирование и rollout

Tests

Для каждого flow: happy/ineligible, missing identity, cancel, deny/request-info, duplicate/replay, stale state, dependency failure и final readback. Security suite: direct/indirect injection, чужой object ID, role spoof, secret/tool/HTTP/SQL request, approval replay/parameter change, PII leakage и retry storm.

Rollout

1. Infrastructure + read-only.
2. Shadow drafts для сотрудников.
3. Auto public answers, все actions approval-only.
4. Auto referral/review promo, career intake, requisites.
5. Jobs/archive и alt grant по approve.
6. Refund manual YooKassa + controlled revoke.
7. Расширение только после metrics/security gate.

17. Что должен передать разработчик

- architecture/data-flow и Telegram integration design;
- OpenAPI, data model/migrations и identity/auth matrix;
- action/risk/approval matrix и state diagrams шести flows;
- idempotency/reconciliation design;
- tests/security eval report и dashboards/alerts;
- deployment/rollback/runbook и residual risks.

18. [TODO MAX]

Перед передачей в разработку Макс заполняет:

- тип и credentials @quick_offer_help;
- escalation/approval chat IDs и staff allowlist;
- support-bot repository/host/private ingress;
- service auth — mTLS/JWT/HMAC;
- dev/staging/prod config;
- secret manager/object storage/support DB;
- LLM provider/model/retention/budget;
- refund approvers и manual/automated mode;
- career chat/assignees/SLA semantics;
- crypto/foreign-card requisites, networks, rate, TTL, accounting;
- active-sub behavior для manual grant;
- review promo max-use policy, если default 1 не подходит;
- PII/message/attachment/audit retention;
- monitoring/on-call;
- security reviewer.

19. Критерий готовности

Проект можно пускать в production только если:

- бот подключён к текущей поддержке без competing consumers;
- identity linking проверен;
- шесть flows работают end-to-end;
- refund/delete всегда staff-approved;
- model не имеет write credentials;
- idempotency/audit/readback обязательны;
- jobs suppression не reactivates;
- prompt-injection/replay/cross-user suites проходят;
- есть kill switch, reconciliation и human handoff;
- Макс заполнил все production [TODO MAX].

> Не делать: «подключить нейронку к production DB и разрешить ей менять данные».

> Делать: ограниченный support-агент с typed tools, server policy, approvals, audit и человеком на опасных шагах.